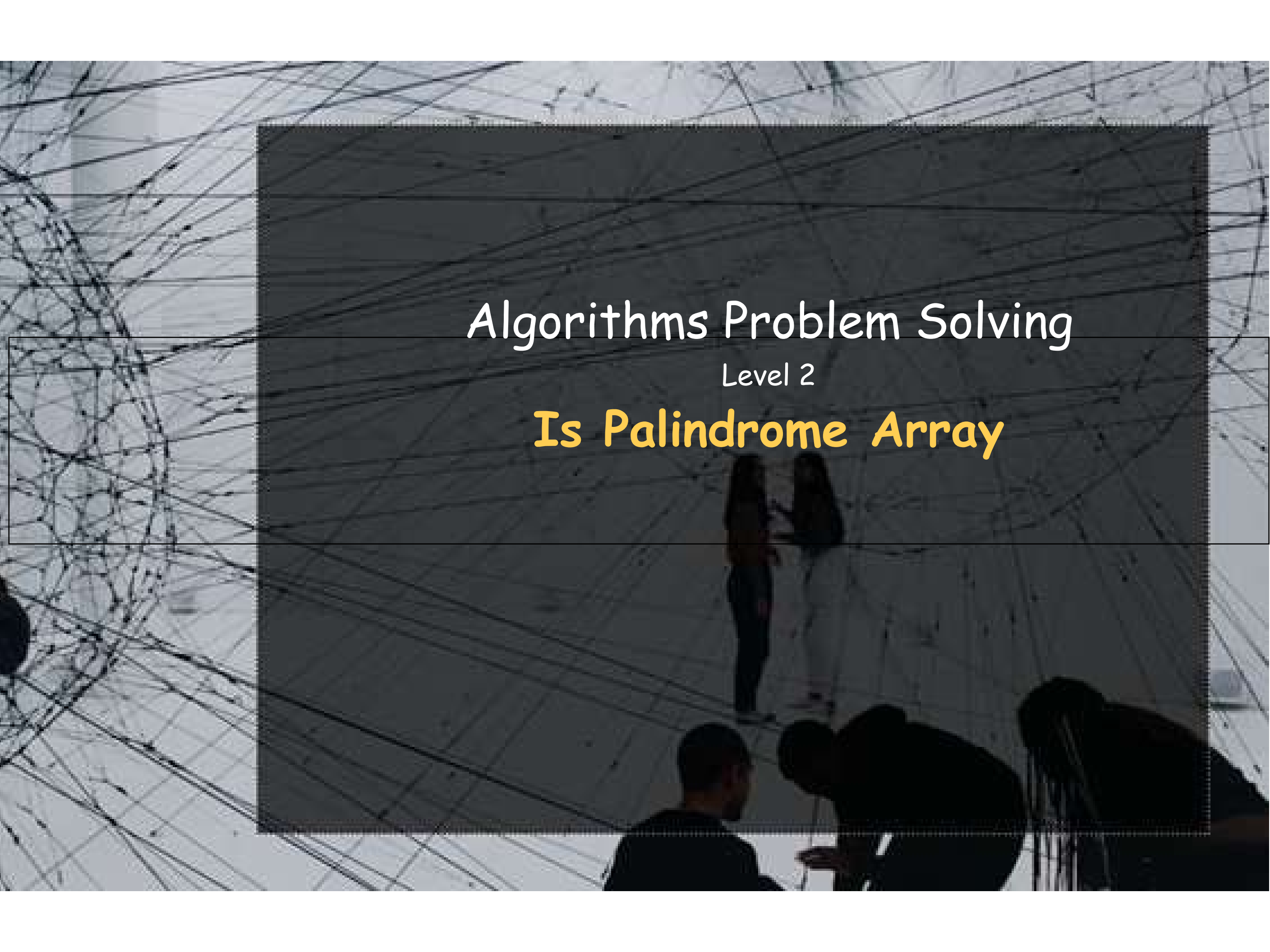


Algorithms Problem Solving Level 2

Is Palindrome Array



Algorithms Problem Solving

Level 2

Is Palindrome Array

Definition

- ◆ This program allows the user to enter numbers into an array.
- ◆ It then checks whether the array is a Palindrome array.
- ◆ A Palindrome array reads the same from left to right and from right to left.
- ◆ The first element must be equal to the last element.
- ◆ The second element must be equal to the second-last element, and so on.

Explanation

- ◆ `IsPalindromeArray()` compares elements from both ends of the array.
- ◆ The important comparison is:
if (`arr[i] != arr[length - i - 1]`)
- ◆ `i` starts from `0`, so `arr[i]` represents the element from the beginning.
- ◆ `length - i - 1` calculates the matching position from the end.
- ◆ For an array with 6 elements:
Index: 0 1 2 3 4 5
Array: 10 20 30 30 20 10
- ◆ The comparisons are:
`arr[0] ↔ arr[5] → 10 = 10`
`arr[1] ↔ arr[4] → 20 = 20`
`arr[2] ↔ arr[3] → 30 = 30`
- ◆ If any pair is different, the function immediately returns `false`.
- ◆ If all pairs match, the function returns `true`.

Steps in main()

1. Create an array with a maximum size of `100`.
2. Set `arrLength` to `0`.
3. Ask the user to enter a number.
4. Add the number to the array using `AddElementToArray()`.
5. Ask whether the user wants to add another number.
6. Repeat until the user chooses `0`.
7. Print the array elements.
8. Call `IsPalindromeArray()`.
9. Compare elements from the beginning and the end.
10. If all matching pairs are equal, return `true`.
11. If any pair is different, return `false`.
12. Print the result.

💡 Example

◆ Input array:

10 20 30 30 20 10

◆ The program compares:

10 ↔ 10

20 ↔ 20

30 ↔ 30

◆ All corresponding elements are equal.

◆ Therefore, the array is a Palindrome array.

➡ Result

Array Elements: 10 20 30 30 20 10

Yes array is Palindrome

✚ Note

- ◆ The expression:
 $\text{length} - i - 1$
 is important because the last valid index is always ``length - 1``.
- ◆ The function could stop after checking only half of the array because each comparison checks two elements at once.
- ◆ For example, with 6 elements, only 3 comparisons are actually necessary.
- ◆ The current loop checks the entire array:
 `for (int i = 0; i < length; i++)`
- ◆ This still produces the correct result, but it repeats the same comparisons in reverse order.

⊘ Warning

- ◆ The program should ensure that no more than `100` elements are added to the array.
- ◆ `ReadNumber()` only checks that entered numbers are positive.
- ◆ `cmath` and `corecrt\_math\_defines.h` are unnecessary in this program.
- ◆ The wording `No array is not Palindrome` would be clearer as `No, array is not Palindrome`.



Comparison

▨ Palindrome Array

- ✓ Reads the same from both directions.
- ✓ First element equals last element.
- ✓ Second element equals second-last element.
- ✓ Example: `10 20 30 30 20 10`.

▨ Not Palindrome Array

- ✓ At least one pair of corresponding elements is different.
- ✓ First and last elements may be different.
- ✓ The function returns `false`.
- ✓ Example: `10 20 30 40 20 10`.

✦✦ Summary

- ✓ Store numbers in an array.
- ✓ Compare the first element with the last.
- ✓ Compare the second element with the second-last.
- ✓ Continue toward the center.
- ✓ If all pairs match, return `true`.
- ✓ If any pair is different, return `false`.
- ✓ A Palindrome array reads the same from left to right and right to left.

Thank you 😊